THE UNIVERSITY OF QUEENSLAND AUSTRALIA | CREATE CHANGE

REIT6811: Git basics

Dr Alina Bialkowski

Software Version Control

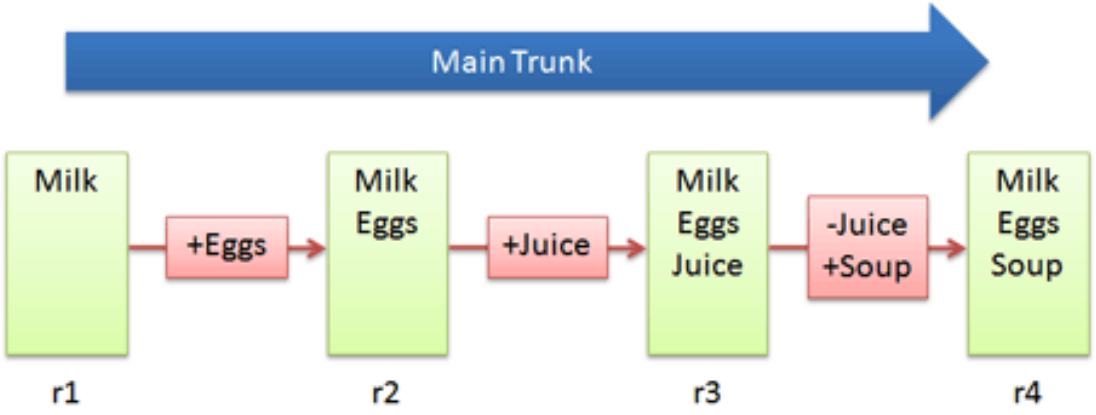
Version control is a way of tracking the change history of a project

A type of “database” (repository) that records changes to files over time and allows you to recall specific versions later

Many types including: Git, SVN, CVS

Features:

- Complete long-term change history of every file
- Branching and merging
- Traceability



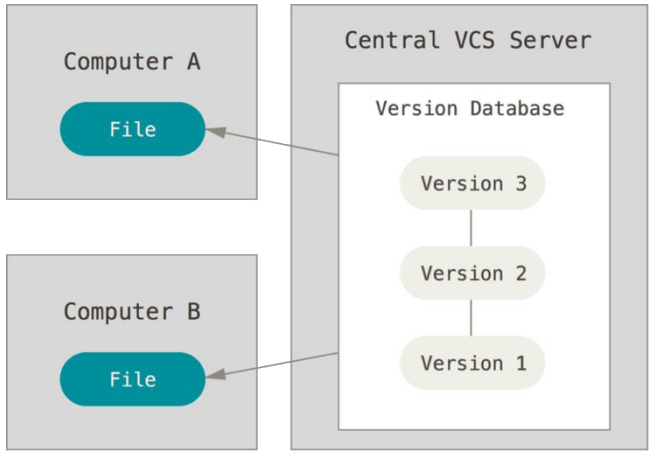
Centralised vs Distributed

Advantages:

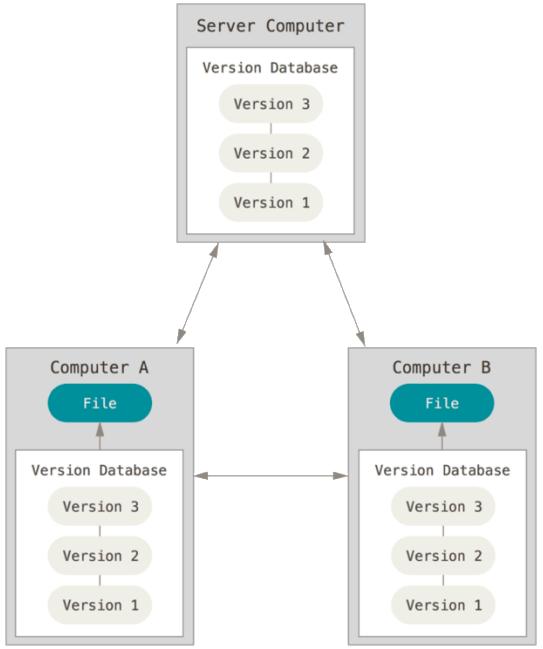
- Administrators have fine-grained control
- Easier to administer than dealing with local databases on every client

Disadvantages:

- Single point of failure
- If server goes down no one can collaborate or save versioned changes
- If hard disk becomes corrupted, you could lose everything



- Advantages:
  - Every client has full repository including its full history
  - Can commit even when not connected to the server
- Disadvantages:
  - Less administrator control



Git

The most widely used modern version control system in the world[1]

Developed in 2005 by Linus Torvalds (the creator of the Linux operating system kernel)

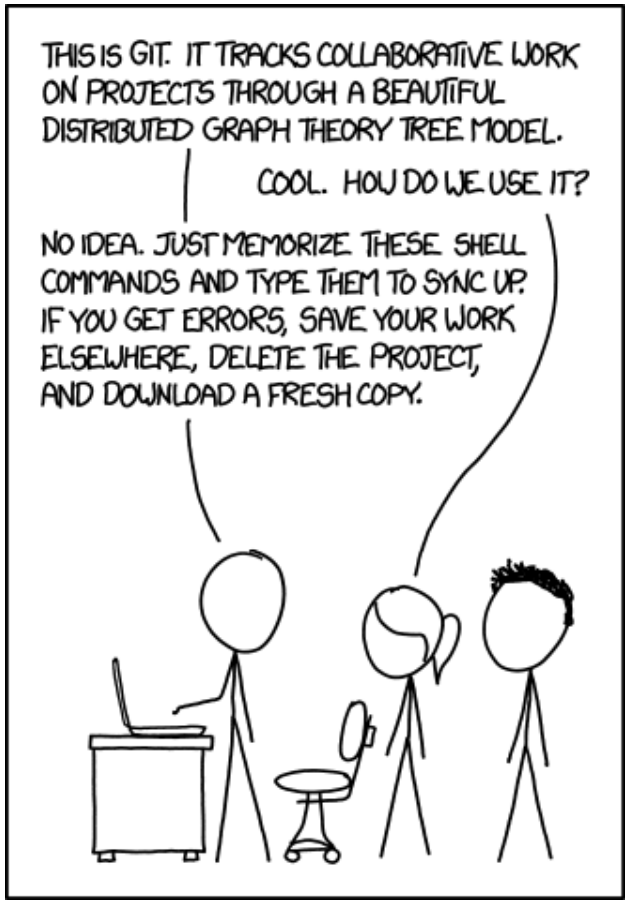
Free & open source

Decentralised/distributed version control system

Online repository hosting websites: GitHub, GitLab, and BitBucket

Documentation:

- Official: https://git-scm.com/doc
- Bitbucket: https://www.atlassian.com/git/tutorials
- GitLab: https://docs.gitlab.com/ee/README.html
- GitHub: https://guides.github.com/activities/hello-world/

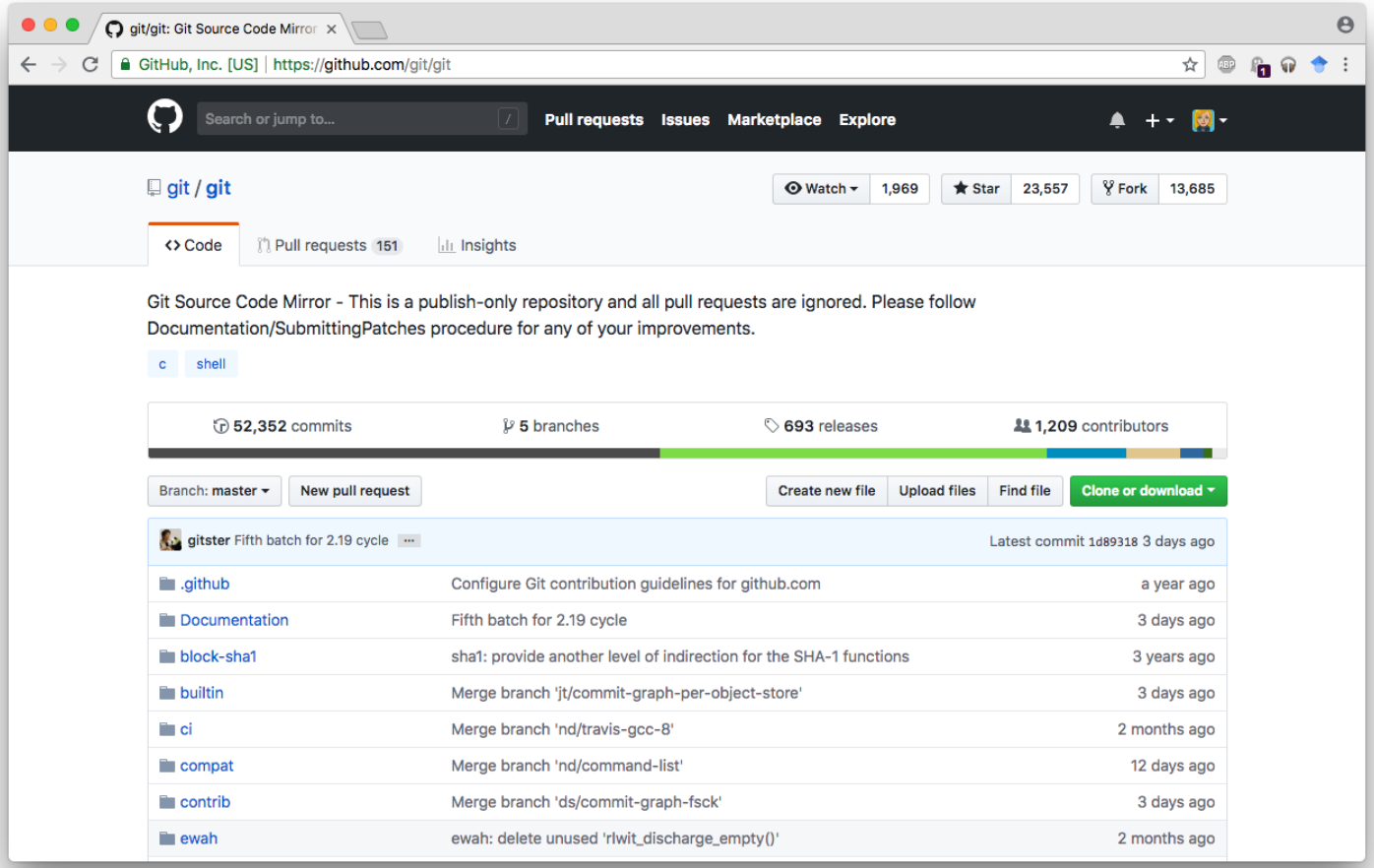


https://xkcd.com/1597/

[1]: https://www.atlassian.com/git/tutorials/what-is-git

Example Git Repository

Git itself is open source and hosted on GitHub: https://github.com/git/git



Git Installation

Operating System Specific

- Windows: Git for Windows installer
- Mac: Install Xcode Command Line Tools
- Linux: Install through distribution package management (e.g. sudo apt-get install git)

Check that it is installed:

```
$ git --version
git version 2.43.0
```

Configure:

```
$ git config --global user.name "Alina Bee"
$ git config --global user.email alinabee@example.com
```

Git Areas and File States

Different areas of Git:

- Working directory; Staging area; Local repository; Remote repository

States your files can be in:

- Modified: you have changed the file but have not committed it to your database yet
- Staged: you have marked a modified file in its current version to go into your next commit snapshot
- Committed: data is safely stored in your local database
- Pushed: data is safely stored in your remote (server) database

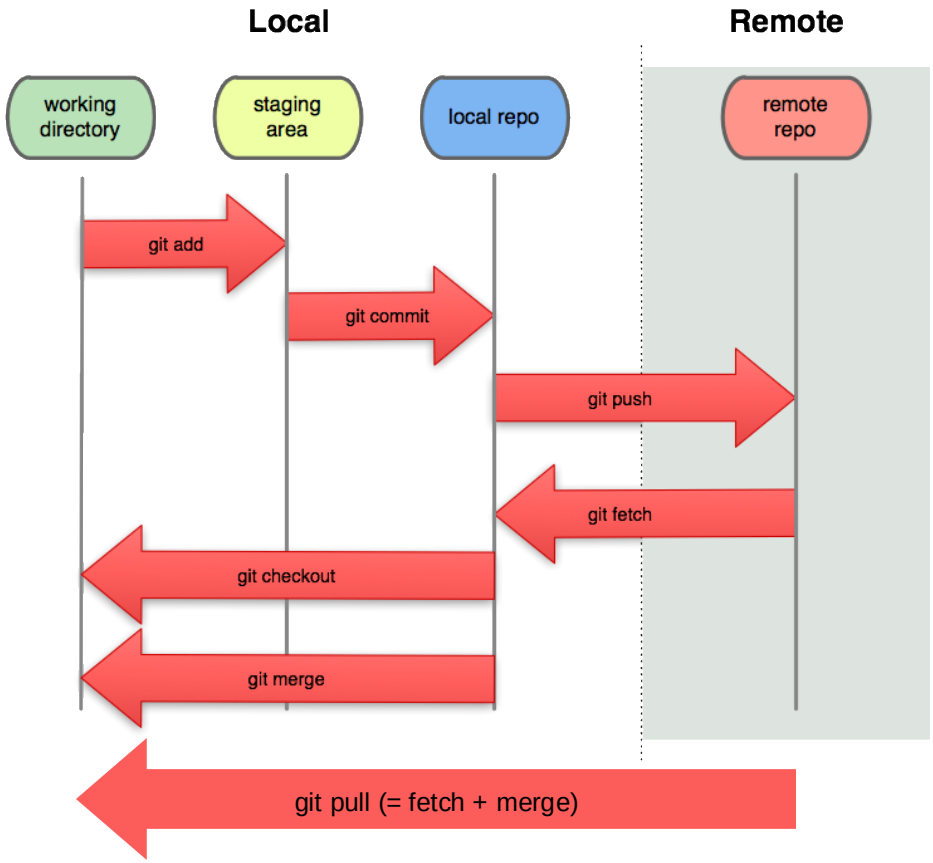


Image from: https://github.com/datasciencelabs/2016/blob/master/lectures/git-and-github/version-control.Rmd

Interacting with Git

Can interact with Git through command-line tools and various graphical user interfaces

We'll go through command-line tools:

- Allows access to **all** Git commands (most GUIs implement only a subset)
- If you know the command-line version you can figure out the GUI version
- Access through Command Prompt ('cmd') in Windows; Terminal in Linux/Mac

Image from: https://codehangar.io/git-gui-vs-command-line/

Setting up a Repository

Two main ways:

- 1. Take a local directory that is currently not under version control and turn it into a Git repository
- 2. You can *clone* an existing Git repository from elsewhere

1. Initialising a repository in an existing directory

```
$ cd /path/to/my/codebase
$ git init
$ git add .
$ git commit -m "Initial commit"
```

2. Cloning an existing repository

```
$ git clone git@github.com:foo/bar.git
$ cd bar
```

Other Good Practices

Set up a .gitignore file

- This will prevent unwanted files from being added to the repository
- e.g. compiled code, large frequently changing binaries

```
- *.so
- *.dylib
- *.dll
- *.exe
- *.out
- build/
```

Create a README.md file

- This provides an overview of the project, and shows up on the front page of online graphical Git packages like GitHub/GitLab/BitBucket
- It is written in the “Markdown” language, great cheatsheet here: https://github.com/adam-p/markdown-here/wiki/Markdown-Cheatsheet
  - ‘#’ is for headings; ‘-‘ is for dot points

File Status

Check the status of your files (lists all new or modified files to be committed)

```
git status
```

Tracking new files

```
git add [filename]
```

Checking the differences between files not yet staged

```
git diff
```

Checking the differences between staged files and last file version

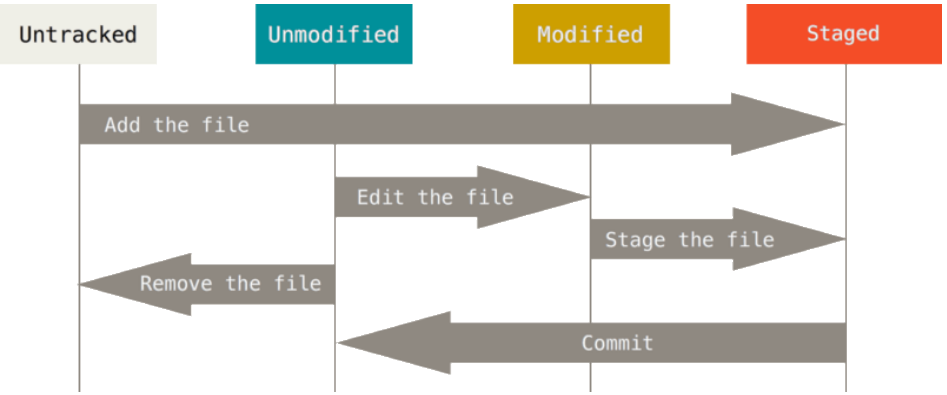
```
git diff --staged
```

Committing your changes

```
git commit -m "[descriptive message]"
```

Remove files from being tracked:

```
git rm bad_file.txt
```



Much nicer in graphical format!:

Image source: https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository

Git - Saving Changes to Repository

`git commit -m "[descriptive message]"`

- Stores the current contents of the staged index in a new commit along with a log message from the user describing the changes.

`git checkout [filename]`

- Replace the current filename in the working directory with the file in the local repository

`git push`

- Sends committed changes to a remote repository.

`git pull`

- Incorporates changes from a remote repository into the current branch. In its default mode, `git pull` is shorthand for `git fetch` followed by `git merge FETCH HEAD`

`git log`

- Print out a log of all commit messages

https://xkcd.com/1296/

Branching & Merging

Branching:

```
git branch
```

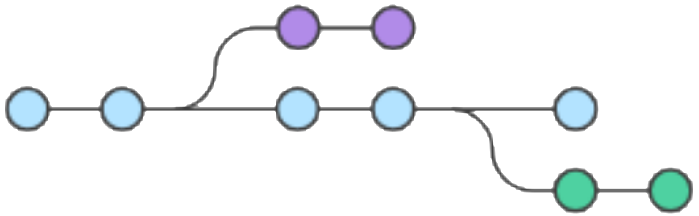
- Lists all the branches

```
git branch <branch_name>
```

- Create a new branch called <branch_name>. This does not check out the new branch

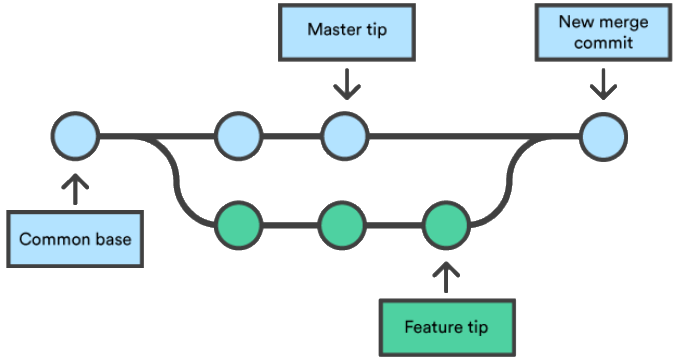
```
git branch -d <branch_name>
```

- Delete the specified branch.



Merging:

```
# Start a new feature
git checkout -b new-feature master
# Edit some files
git add <file>
git commit -m "Start a feature"
# Edit some files
git add <file>
git commit -m "Finish a feature"
# Merge in the new-feature branch
git checkout master
git merge new-feature
git branch -d new-feature
```



Images from: https://www.atlassian.com/git/tutorials/using-branches/git-merge